GPU-accelerated Plane Graph Region Extraction Algorithms

Hatem Mira[a,*], Cherif Salama[a,**]

[a]Department of Computer Science and Engineering, The American University in Cairo, Cairo, Egypt

ARTICLE INFO

Keywords:
Plane Graph
Region Extraction
Computational Geometry
Meshing
GPU
CUDA
CAD

ABSTRACT

This paper presents two high-performance GPU-accelerated algorithms for the extraction of connected regions in plane graphs implemented using the CUDA programming model. While sequential algorithms for extracting all regions of a plane graph $G(V, E)$, with m edges, achieve an optimal time complexity of $O(m \log m)$, they remain constrained by CPU throughput when processing large-scale datasets, and although established libraries such as the Boost Graph Library and the Computational Geometry Algorithms Library provide robust CPU-based solutions, a dedicated GPU implementation currently does not exist in the literature. We introduce a parallel framework that leverages GPUs to exploit the massive parallelism inherent in edge sorting and face traversal. Experimental results demonstrate that our fastest CUDA-based approach significantly outperforms state-of-the-art CPU implementations, achieving a speedup of up to 33× in total computation time.

1. Introduction

A plane graph is a specific topological embedding of a planar graph in a two-dimensional Euclidean space, where vertices are mapped to distinct coordinates and edges are mapped to continuous arcs that intersect exclusively at their common endpoints. Identifying and extracting the discrete regions (or faces) bounded by these edges is a fundamental operation across an extensive array of domains, including geographic information systems (GIS), computer graphics, VLSI layout verification, mesh generation, and automated robotics navigation [2, 6, 13]. In modern computational engineering, simulations involving finite element analysis or fluid dynamics frequently manipulate structural meshes containing tens of millions of distinct elements [11, 12, 4]. At this scale, serial or poorly parallelized geometric extraction routines quickly become severe computational bottlenecks.

Two primary optimal sequential algorithms for region extraction are discussed in this paper. The first one [10] is based on wedges, while the second one [5] is based on half-edges (HEs) and is implemented in many computational geometry libraries, such as the Computational Geometry Algorithms Library (CGAL) [8] and the Boost Graph Library (BGL) [1]. In this paper, these approaches are referred to as the Wedge-based and HE-based algorithms, respectively. Both methods achieve a time complexity of $O(m \log m)$ and a space complexity of $O(m)$, where m denotes the number of edges in the graph $G(V, E)$. As shown by Euler's formula for planar graphs, a simple plane graph with $n \geq 3$ vertices has at most $m \leq 3n - 6$ edges [7], confirming that such graphs are inherently sparse.

Although theoretically optimal, the original sequential algorithms can be computationally impractical when applied to large-scale graphs commonly encountered in modern scientific and engineering applications, and the emergence of massively parallel platforms, such as GPUs with NVIDIA's CUDA [14] framework, creates new opportunities for accelerating algorithmic graph operations.

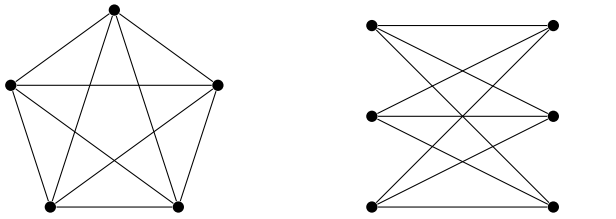
This paper introduces comprehensive GPU-accelerated parallel adaptations of both the Wedge-based and the Half-Edge-based region extraction algorithms. We evaluate our parallel CUDA C++ implementations against established CPU geometric frameworks, including CGAL and BGL, as well as heavily optimized sequential baselines. Our parallel designs adapt a parallel pointer-jumping (path doubling) [9] technique and leverage the parallel primitives of the Thrust library [3]. We utilize a suite of synthetic 2D mesh datasets to simulate the large-scale planar graph structures encountered in modern engineering. Experimental results demonstrate that our GPU implementations maintain absolute correctness while achieving substantial speedups—up to 33× in our tests—compared to CPU baselines. These findings highlight the effectiveness of parallelization for geometric graph processing and demonstrate the practicality of our approach for large-scale applications.

The remainder of this work is structured as follows. Section II formalizes the necessary mathematical definitions and structural notations. Section III outlines the operational logic of the baseline sequential algorithms. Section IV provides a deep overview of our proposed parallelization methodologies and CUDA implementation strategies. Section V presents the experimental results, performance characteristics, and workload profiling, and Section VI concludes the paper.

2. Definitions and Notations

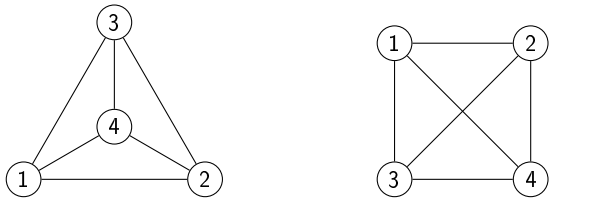
Planar Graph: A graph $G = (V, E)$ is defined as *planar* if it can be drawn on the Euclidean plane $\mathbb{R}^2$ without any edges crossing except at their endpoints (vertices). The term describes a topological property of the graph itself rather than a particular drawing of it. In other words, a graph is

*Corresponding author
**Principal corresponding author
hmira@aucegypt.edu (H. Mira); cherif.salama@aucegypt.edu (C. Salama)
ORCID(s): 0009-0004-0238-4922 (H. Mira); 0000-0002-6986-4697 (C. Salama)



(a) The complete graph K_5 (b) The bipartite graph $K_{3,3}$

Figure 1: The two minimal non-planar graphs shown with unavoidable edge crossings.



(a) A plane embedding of K_4 (b) A non-plane drawing of K_4

Figure 2: Verification of the planarity of K_4: (a) provides a valid plane embedding, while (b) illustrates a drawing with a crossing that does not alter the graph's underlying planarity.

called planar if there exists at least one way to represent it in the plane so that no two edges intersect improperly.

It is crucial to distinguish the graph's inherent planarity from its visual representation; a planar graph may be rendered with edge crossings in a given drawing, yet it remains planar as long as at least one crossing-free configuration exists. Conversely, *non-planar graphs* cannot be drawn without at least one intersection, regardless of the arrangement. The most fundamental examples of non-planar graphs are the complete graph K_5 and the complete bipartite graph $K_{3,3}$ shown in Figure 1.

Plane Graph: While "planar" refers to the capacity of the graph to be drawn without crossings, a *plane graph* refers to a specific *planar embedding* (or drawing) of that graph in which no two edges intersect except at their endpoints.

As illustrated by the complete graph K_4 in Figure 2, a single planar graph can yield multiple representations. A drawing of K_4 in a standard rectilinear format typically results in an edge crossing; however, by relocating a single vertex or routing an edge externally, a crossing-free "plane" representation is achieved. This demonstrates that while K_4 is fundamentally a planar graph, only specific drawings of it constitute plane graphs.

A plane graph divides the plane into distinct areas called *regions* (also known as *faces*, *windows*, or *meshes*). Each region is defined by the set of edges forming its boundary. For example, the graph shown in Figure 3 contains three such regions:

$$R_1 = ((v_1, v_2), (v_2, v_4), (v_4, v_5), (v_5, v_6), (v_6, v_1))$$
$$R_2 = ((v_1, v_6), (v_6, v_3), (v_3, v_1))$$

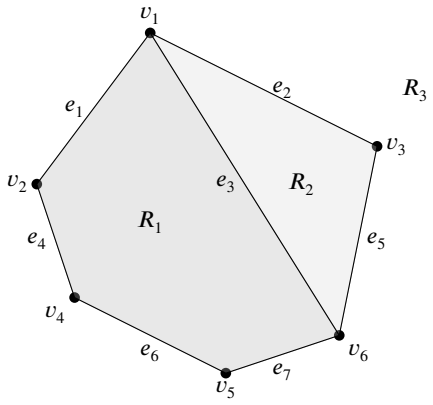


Figure 3: Plane graph with seven edges forming three regions.

$$R_3 = ((v_1, v_3), (v_3, v_6), (v_6, v_5), (v_5, v_4), (v_4, v_2), (v_2, v_1))$$

Among them, R_3 is called the *exterior region*, which lies outside the boundary of the graph. Given two edges $e_1 = (v_i, v_j)$ and $e_2 = (v_j, v_k)$; if rotating e_1 in a clockwise direction around the shared vertex v_j toward e_2 does not encounter any other edge incident on v_j, then the area enclosed between e_1 and e_2 defines a *wedge*, denoted (v_i, v_j, v_k). Two wedges such as (v_i, v_j, v_k) and (v_j, v_k, v_l) are called contiguous. In any plane graph with m edges, there are exactly $2m$ wedges. Returning to the graph in Figure 3, which has 7 edges, we find 14 wedges:

$$(v_2, v_1, v_3),\ (v_3, v_1, v_6),\ (v_6, v_1, v_2),\ (v_4, v_2, v_1),$$
$$(v_1, v_2, v_4),\ (v_1, v_3, v_6),\ (v_6, v_3, v_1),\ (v_2, v_4, v_5),$$
$$(v_5, v_4, v_2),\ (v_4, v_5, v_6),\ (v_6, v_5, v_4),\ (v_1, v_6, v_3),$$
$$(v_3, v_6, v_5),\ (v_5, v_6, v_1)$$

Each region in a plane graph can be represented in any one of the three equivalent ways:

- As a sequence of *vertices*
- As a sequence of *edges*
- Or as a sequence of *wedges*.

For example, region R_1 from Figure 3 can be expressed as:

$$(v_1, v_2, v_4, v_5, v_6),$$
$$((v_1, v_2), (v_2, v_4), (v_4, v_5), (v_5, v_6), (v_6, v_1)),\ \text{or}$$
$$((v_1, v_2, v_4), (v_2, v_4, v_5), (v_4, v_5, v_6), (v_5, v_6, v_1),$$
$$(v_6, v_1, v_2))$$

The three representations are equivalent and can be easily transformed into each other. Thus, either of them can be chosen for convenience.

Half-Edge: In the context of computational geometry, a half-edge is a directed representation of an edge. It is like a general directed edge, but additionally, it points to the next half-edge, which allows a "walk" around any polygon.

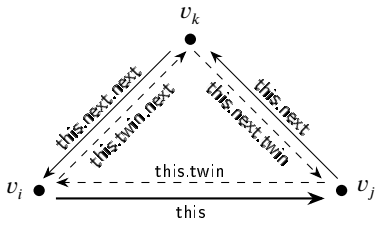


Figure 4: An illustration of half-edges and their twin edges of a triangle

Twin Edge: Every half-edge $e = \langle u, v \rangle$ that originates at the vertex u and ends at the vertex v is associated with a unique *twin edge* $\bar{e} = \langle v, u \rangle$. The twin edge originates at v and ends at u, representing the same geometric span but oriented in the opposite direction. Figure 4 illustrates half-edges and their twin edges of a triangle.

3. Optimal Sequential Algorithms

In this section, we illustrate the steps of the optimal sequential algorithms for region extraction (the Wedge-based and the HE-based), which have run-time complexity $O(m \log m)$ and space complexity $O(m)$. We will also shed light on the time complexity of each step because it will be the guide to the design of the GPU-accelerated algorithms proposed in the methodology. Both algorithms take as input an undirected connected graph $G(V, E)$ with a list of vertices V, each vertex v_i with coordinates (x_i, y_i), and a list of undirected edges E.

3.1. The Wedge-based Algorithm

The algorithm proceeds in two main phases: first, it identifies all possible wedges; next, it groups these wedges to form the regions.

3.1.1. Phase 1: Wedge Construction

The first phase identifies all wedges in the input plane graph, which are angular regions formed around each vertex by consecutive incident edges. The steps of this phase are as follows:

1. **Edge Duplication:** Each undirected edge (v_i, v_j) is duplicated to form two directed edges $\langle v_i, v_j \rangle$ and $\langle v_j, v_i \rangle$ representing both directions. This prepares the graph for directional angular computations. Time complexity to duplicate all edges is $O(m)$.
2. **Angle Calculation:** For each directed edge $\langle v_i, v_j \rangle$, the angle θ relative to the positive side of the horizontal axis passing through its source vertex is computed, enabling sorting of edges by their spatial orientation around each vertex. The time complexity of calculating angles for all edges is $O(m)$.
3. **Sorting:** Directed edges $\langle v_i, v_j \rangle$ are first sorted by their source vertex v_i and then by their angles θ. This ensures that edges incident to each vertex are arranged in a counter-clockwise order to allow wedge formation. The time complexity of sorting edges is $O(m \log m)$. An example of sorting edges by their angles is shown in Figure 5, where the sorted edges are e_i, e_j, e_k.
4. **Wedge Generation:** For each vertex group which has the same source vertex v_i, wedges are constructed by pairing each edge $(\langle v_i, v_j \rangle, \theta_1)$ with its successive neighbor in the group $(\langle v_i, v_k \rangle, \theta_2)$ to build the wedge (v_k, v_i, v_j), with the last edge paired with the first to complete the cycle. The time complexity of building wedges from the edges is $O(m)$. In Figure 5 the constructed wedges are (v_6, v_1, v_2) (v_3, v_1, v_6) (v_2, v_1, v_3).

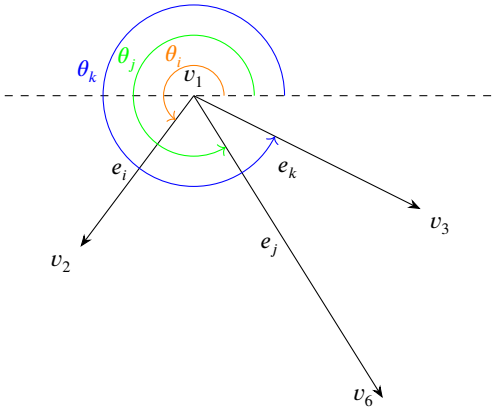


Figure 5: Sorting Edges according to angle with the horizontal axis. Sorted edges are e_i, e_j, e_k

The time complexity of the first phase is $O(m \log m)$ because of the sorting step.

3.1.2. Phase 2: Build Regions from Wedges

The region grouping phase assembles wedges into closed polygonal regions, identifying all face boundaries of the plane graph. It can be easily proven that a region can be defined by a sequence of wedges $(W_1, W_2, ..., W_k)$ if and only if W_i and W_{i+1} are contiguous for $i = 1, 2, ..., k-1$ and also W_k and W_1 are contiguous. We will depend on that to create the regions according to the following steps:

1. **Sorting Wedges:** Given the list of wedges (v_i, v_j, v_k) constructed from the first phase, wedges are sorted by v_i then by v_j. Time complexity is $O(m \log m)$.
2. **Marking Wedges:** Mark all wedges as unvisited wedges. Time Complexity is $O(m)$.
3. **Region Start Wedge:** Each wedge is examined in order. If a wedge has not yet been used to construct a previous region, it is used as the starting wedge W_1 for constructing a new region, and it is marked as a visited wedge.
4. **Contiguous Wedge Search:** In this step, we iterate over the wedges till we define the current region as follows:
   (a) **Binary Search:** Using binary search within the sorted wedge list, the algorithm finds the next contiguous wedge W_{i+1} sharing an edge with the current one W_i, extending the region boundary,

and the new wedge is marked as a visited wedge. Time complexity of the binary search step is $O(\log m)$.

(b) **Closure Detection:** The process continues until the initial wedge is reached again, completing the closed polygonal region.

5. **Completion:** If there is no unvisited wedge, then the phase is completed; otherwise, go back to step 3.

Because every wedge is visited exactly once during the tracing loops, the cumulative cost of the binary searches across all $2m$ wedges is mathematically bounded by $O(m \log m)$. Consequently, when combining the sorting stage with the traversal loop, Phase 2 maintains an overall time complexity of $O(m \log m)$, matching the total asymptotic bounds of the complete extraction algorithm.

3.2. The HE-based Algorithm

The algorithm proceeds in two main phases: first, it constructs the half-edges with their next and twin edges from the input graph; next, it traverses the half-edges to extract all regions.

3.2.1. Phase 1: Half-Edges Construction

The first phase builds the topological relationships required for the half-edges. The steps are as follows:

1. **Half-Edge Creation:** Each undirected edge (v_i, v_j) is split into two directed *half-edges*, $e_{i,j}$ (originating at v_i) and $e_{j,i}$ (originating at v_j). Each half-edge is assigned to its twin edge immediately. Time complexity is $O(m)$.
2. **Angle Calculation:** For each directed edge $\langle v_i, v_j \rangle$, the angle θ is calculated in a similar way to the step in the Wedge-based algorithm. The time complexity of calculating angles for all edges is $O(m)$.
3. **Angular Sorting:** For every vertex v, all half-edges originating at v are sorted by their polar angle ascending. This step is identical to the Wedge-based algorithm, and is required to determine the relative order of edges around a vertex. Time complexity is $O(m \log m)$.
4. **Next Edge Linking:** For each half-edge $e_{i,j}$ (which terminates at v_j), its next edge must be a half-edge originating at v_j. Specifically, the immediate clockwise successor of its twin edge around the vertex v_j (the previous half-edge of its twin edge in the sorted half-edges around v_j). This ensures that the traversal follows the boundary of the incident face. The time complexity of linking one edge is $O(1)$ and, hence, for all edges is $O(m)$. A visual example of this step is shown in Figure 6, where the next half-edge of $e_{3,1}$ is calculated as $e_{1,6}$ because it is the half-edge that has the largest angle less than its twin edge $e_{1,3}$.

Similar to the Wedge-based approach, the total time complexity of the HE-based algorithm is $O(m \log m)$ due to the sorting requirement.

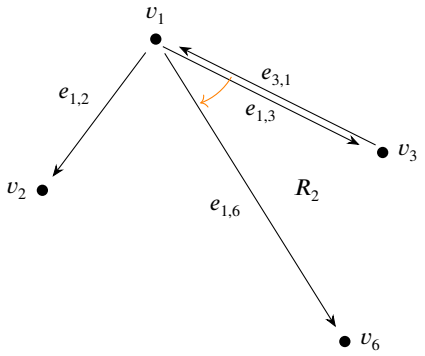


Figure 6: Next Edge Linking. The next edge of $e_{3,1}$ points to the half-edge $e_{1,6}$ based on the angular order around v_1.

3.2.2. Phase 2: Region Traversal

Once the half-edges with their next and twin edges are calculated, extracting regions becomes a linear-time pointer-following exercise. Unlike the Wedge-based algorithm, no binary search is required in this phase because the topology is explicitly stored.

1. **Initialization:** All half-edges are marked as unvisited. A global list of regions is initialized. Time complexity is $O(m)$.
2. **Edge Selection:** The algorithm iterates through the half-edges. If a half-edge e has not been visited, it is selected as the starting edge of a new region.
3. **Cycle Traversal:** Starting from e, the algorithm follows the next half-edges until the starting half-edge e is reached again. All half-edges encountered in this cycle are marked as visited and added to the current region.
4. **Completion:** The process repeats until all half-edges have been visited.

Since each half-edge is visited exactly once and the time complexity is constant per helf-edge during the traversal, the time complexity of Phase 2 is $O(m)$. Consequently, the overall complexity of the HE-based algorithm is dominated by the sorting in Phase 1, resulting in $O(m \log m)$. Although the HE-based algorithm has the same time complexity as the Wedge-based algorithm, $O(m \log m)$, it does not have the binary search step, and it is expected to have a better run-time performance.

4. Methodology

In this section, we will illustrate the GPU-accelerated algorithm for both the wedge-based and the HE-based algorithms.

4.1. The Wedge-based Algorithm

The CUDA-based parallel implementation of the wedge-based algorithm adheres to the two-phase structure originally introduced by Jiang and Bunke [10], with necessary adaptations to enable parallel execution. These adaptations

include the incorporation of CUDA-specific libraries and the application of parallelization strategies tailored for GPU architectures. The implementation of each phase is detailed as follows:

4.1.1. Phase 1: Wedge Construction

The first phase identifies all wedges in the input plane graph. Despite this phase being computationally intensive, it is highly parallelizable and hence well-suited for GPU execution with the use of the Thrust [3] library in CUDA. The Thrust API provides an easy-to-use high-level interface for parallel programming on CUDA-enabled GPUs. It offers a wide range of functionalities such as scan, sort, reductions, and data transformations. The computation proceeds as follows:

- **Copy Edges:** Copy the edges data from the CPU to the GPU.
- **Edge Duplication:** Each undirected edge is passed to a thread to get duplicated to form two directed edges representing both directions. This prepares the graph for directional angular computations. This step is fully parallelizable in a direct way by duplicating each edge on a thread.
- **Angle Calculation:** Each directed edge is passed to a thread, then the angle relative to the horizontal axis passing through its source vertex is calculated. This step is fully parallelizable in a direct way by calculating the angle for each edge on a thread.
- **Sorting:** Directed edges are first sorted by their source vertex and then by their angles. This step is parallelized using the sort function in the Thrust library.
- **Grouping Edges:** The sorted edges are grouped by their source vertices. This step is parallelized using the reduce_by_key function in the Thrust library in CUDA, which identifies the groups around each vertex and calculates their sizes.
- **Wedge Generation:** Each vertex group is passed to a thread where wedges are constructed by pairing each edge with its successive neighbor in the group, with the last edge paired with the first to complete the cycle. This step is also parallelizable, with each thread independently processing a group of sorted edges, where the number of iterations per thread corresponds to the number of edges in a group, which is typically much less than m, the total number of edges in the graph.

4.1.2. Phase 2: Build Regions from Wedges

The region grouping phase assembles wedges into closed polygonal regions, identifying all face boundaries of the plane graph. Unlike the first phase, this phase in the original algorithm requires sequential traversal and dynamic data-dependent operations, which are inherently less amenable to parallelization, but still, this phase is accelerated as illustrated in the following steps of the algorithm.

1. **Wedge Sorting:** The resulting wedges from phase 1 are stored by their vertex pairs for efficient lookup. The sort step is done using the Thrust library in CUDA.
2. **Wedge Initialization:** All the wedges are marked as unvisited in preparation for region extraction, each wedge in a thread. This step is fully parallelizable.
3. **Search for the contiguous wedges**: In this step, we pass each wedge to a thread and calculate the contiguous wedge for each wedge using binary search, and save this data. As a result, we can, later, find the contiguous wedge for any wedge in constant time. This step is fully parallelizable as each wedge searches for its contiguous wedge in a thread, and the binary search operation has time complexity $O(\log m)$.
4. **Calculate Leaders**: To uniquely identify the regions of the plane graph, we map all wedges belonging to the same region to a single representative identifier. We employ a parallel pointer jumping technique (also known as path doubling) to achieve this convergence efficiently. Let W be the set of $2m$ wedges. Each wedge $w_i \in W$ maintains a successor pointer $S[i]$ and a label $L[i]$. Initially, $S[i]$ is set to the index of the contiguous wedge of w_i, and $L[i]$ is initialized to the index i. The objective is to reach a steady state where all wedges belonging to the same region $R \subseteq W$ share a uniform label, specifically the minimum index within that region:

$$\forall i \in R, \quad L[i] = \min_{j \in R}\{j\}$$

In a sequential environment, identifying the minimum label in a cycle of length c requires $O(c)$ operations. To leverage GPU parallelism, we utilize a geometric progression managed by a host-side loop. In each iteration k, a kernel is launched where every wedge is processed by a thread to concurrently update its label and successor by referencing those of its current target:

$$S_{k+1}[i] = S_k[S_k[i]]$$

$$L_{k+1}[i] = \min(L_k[i], L_k[S_k[i]])$$

This doubling effect ensures that by iteration k, each wedge has traversed a path of length 2^k. Consequently, any region of size c is fully traversed in $\lceil \log_2 c \rceil$ iterations. Simultaneously, we perform a reduction-style update on the labels; for each wedge w_i, we compare its current label with the label of its successor $S[i]$ and retain the minimum. The loop terminates when no further label updates occur across the wedges. Figure 7 illustrates the progression of leader values for a region of size 8, with wedge indices ranging from 0 to 7. Initially, each wedge w_i is defined by two properties: $L_0[i] = i$ and $S_0[i] = (i + 1) \pmod 8$. After iteration 1, $L_1[7]$ is the only updated label, as w_7 is the only wedge whose successor has a smaller index. The successors are then updated to $S_1[i] =$

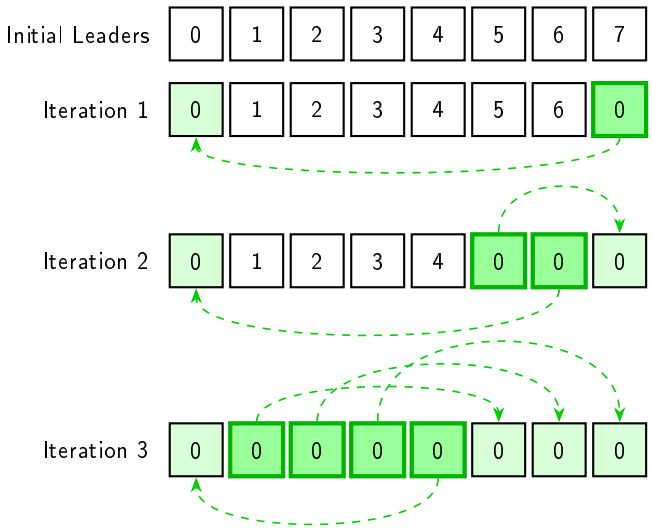


Figure 7: Convergence of the Parallel Pointer Jumping algorithm on an 8-element cycle. The diagram illustrates label propagation (L) where the minimum identifier (0) is distributed across the array. Highlighted cells indicate nodes that adopted the leader's index during the current iteration.

$S_0[S_0[i]] = (i + 2) \pmod 8$. By iteration 2, both w_5 and w_6 update their labels, and the successors for all wedges become $S_2[i] = (i + 4) \pmod 8$. Following the final iteration, all wedges w_0 through w_7 converge to the label 0.

5. **Construct Regions:** Following the convergence of region leaders, a kernel is launched to linearize the constituent wedges of each independent region. In this phase, parallelism is exploited at the region level rather than the wedge level; each thread is assigned to a unique region leader and executes a sequential traversal along the original contiguous pointers, starting from the leader, until the boundary cycle is fully reconstituted.
   While the traversal within each region is sequential, this design minimizes resource utilization by scaling the active thread count to the number of discrete regions ($|R|$) rather than the total number of wedges ($2m$). This reduction in thread density mitigates thread management overhead, prevents GPU resource oversubscription, and maximizes the execution context resources available per active thread.
   Although using pointer jumping would yield a logarithmic time complexity of $O(\log c)$ for a cycle of length c, it requires maintaining a thread allocation scaled to all $2m$ wedges. Consequently, the linear traversal over $|R|$ threads proves more efficient in practice due to reduced kernel launch overhead, even though its execution time remains bounded by the maximum cycle length, $O(\max c)$.
6. **Copying Regions:** Copy the regions data back to the CPU.

In the second phase, the maximum number of iterations executed by any GPU thread during the sequential steps corresponds to the maximum number of wedges in a region of the graph. While this upper bound is $O(m)$, it is typically much smaller in practice.

4.2. The HE-based Algorithm

4.2.1. Phase 1: Half-Edges Construction

All the steps in this phase are fully parallelizable, as the **Half-Edge Creation** and the **Angle Calculation** steps are parallelized through launching kernels for each half-edge, and the **Angular Sorting** step is done using the Thrust library, and the **Next Edge Linking** step is parallelized through kernels for each half-edge. At the end of this phase, the next edge for each half-edge is calculated to be used in the next phase.

4.2.2. Phase 2: Region Traversal

All the steps in this phase are directly parallelized, starting from marking all half-edges as unvisited, then calculating the leader half-edges using the same pointer jumping approach used in the Wedge-based algorithm, and at the end constructing the regions and sending the data back to the CPU.

In this phase, the maximum number of iterations executed by any GPU thread during the sequential steps corresponds to the maximum number of edges in a region of the graph. While this upper bound is $O(n)$, it is typically much smaller in practice.

5. Experiments and Results

5.1. Hardware Description

The experiments were conducted on a high-performance computing node equipped with an Intel Xeon Gold 6548Y+ processor (5th Gen), providing 24 physical cores with a base frequency of 2.50 GHz (up to 4.10 GHz Turbo) and a 1.4 GiB L3 cache. The system features 1.5 TB of DDR5 RAM. Hardware acceleration was provided by an NVIDIA H100 NVL (94GB) GPU.

5.2. Time Measurement Process and Correctness Checking

Timing measurements were performed by invoking the algorithm as a library. The total execution time was measured from immediately before the function call to immediately after its completion, thereby accounting for all host–device data transfer overhead.

To assess both the performance and correctness of the proposed CUDA C++ GPU-based region extraction algorithms, we conducted a comprehensive comparison against a reference CPU implementation based on BGL, CGAL, and also our own C++ implementation of the algorithms.

When performing region extraction using the BGL library, it is necessary to first invoke a planarity testing function (boyer_myrvold_planarity_test) to verify that the input graph is planar prior to extracting its faces. Although extracting the regions from the graph embedding, after calling

the planarity test function, does not take much time, the planarity test itself introduces a considerable runtime overhead, which makes the BGL library not a practical approach when dealing with giant graphs whose planarity is already guaranteed. Therefore, in the Results section, we report only the time required for region extraction after validating the graphs, and only for the smallest graph instances, as including the planarity testing time would not provide a meaningful or practical performance comparison.

In contrast, the CGAL library does not require an explicit external call to a dedicated planarity testing function. Instead, it is designed with robustness and generality in mind, operating without the assumption that the input graph is already connected and planar. While the overall runtime of CGAL is reported, a direct comparison with the GPU-based implementation is not appropriate due to fundamental differences in design objectives and computational models.

Finally, we implemented the Wedge-based and the HE-based algorithms in standard C++ for execution on the CPU. All these implementations serve as a baseline for both correctness verification and runtime analysis, leveraging the prior knowledge that all experimental input graphs are valid connected planar graphs and therefore do not require explicit planarity testing.

5.3. Data

The evaluation was carried out on a suite of synthetic 2D mesh graphs designed to emulate the large-scale meshes commonly found in scientific and engineering applications. These test graphs varied in size and structure, with each graph representing a square grid of $l \times l$ vertices, with l taking values from $\{1001, 4001, 8001, 12001\}$, thereby covering a range from moderately sized to extremely large graphs.

Three distinct edge patterns were used to generate the test graphs, each designed to simulate different types of planar structures. The first pattern (P_1) includes horizontal, vertical, and diagonal edges at most of the vertices, forming a dense mesh characterized by small triangular regions. The second pattern (P_2) is sparser, resulting in a combination of small triangular regions and larger zigzag regions. The third pattern (P_3) represents a hybrid configuration that blends features of the first two, producing regions of varying shapes and sizes, and offering a mid-density mesh.

Across all graph sizes and patterns, the CUDA C++ implementation consistently produced results identical to the CPU version, validating its correctness. Furthermore, it achieved substantial speedup, particularly for larger graphs, demonstrating the benefits of efficient parallelism in handling complex region extraction tasks.

Figures 8, 9, and 10 illustrate the structure of graph patterns P_1, P_2, and P_3, respectively, using a small example with $l = 7$, and Table 1 summarizes the corresponding graph statistics, including the number of vertices, edges, and regions extracted for each pattern.

5.4. Results

For the graph instances listed in Table 1, region extraction was performed using the four CPU-based approaches described above: BGL, CGAL libraries, and our own C++ implementations of the Wedge-based and the HE-based algorithms, and 2 GPU-based approaches, the Wedge-based and the HE-based. Correctness was verified across all graph instances, and the speedup of each implementation was computed as the ratio of the execution time to the HE-based GPU execution time, as the fastest run time results were for the HE-based algorithm either on the CPU or on the GPU for most of the test cases. It is noted that the GPU-accelerated HE-based algorithm is slightly faster than the GPU-accelerated Wedge-based algorithm.

As mentioned previously, for the BGL implementation, timing measurements were limited to the smallest graph group ($l = 1001$), due to the overhead associated with planarity testing. The detailed results are presented in Table 2. Among the CPU approaches, our HE-based C++ implementation achieved better performance than the libraries, as it executes only the core algorithm without performing planarity checks, and better than the Wedge-based implementation, which was expected because it depends on the twin edges to find the next edges, and it does not need to do binary search. Comparing the two parallel approaches shows that the Half-Edge implementation consistently outperforms the Wedge-based framework. This advantage arises because the Half-Edge data structure tracks connectivity natively, avoiding

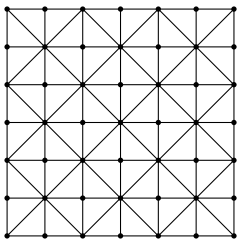


Figure 8: Pattern 1 of graphs for a grid of 7×7 vertices

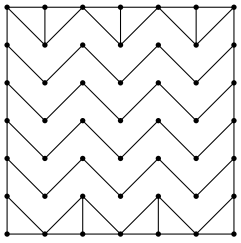


Figure 9: Pattern 2 of graphs for a grid of 7×7 vertices

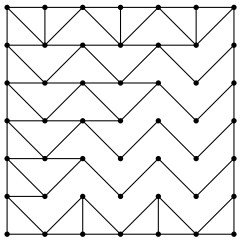


Figure 10: Pattern 3 of graphs for a grid of 7×7 vertices

Graph ID	Pattern Grid Side Length (l)	Pattern Type	Number of Vertices (n)	Number of Edges (m)	Number of Regions
g_1	1,001	P_1	1,002,001	3,002,000	2,000,001
g_2	1,001	P_2	1,002,001	1,004,999	3,000
g_3	1,001	P_3	1,002,001	1,504,499	502,500
g_4	4,001	P_1	16,008,001	48,008,000	32,000,001
g_5	4,001	P_2	16,008,001	16,019,999	12,000
g_6	4,001	P_3	16,008,001	24,017,999	8,010,000
g_7	8,001	P_1	64,016,001	192,016,000	128,000,001
g_8	8,001	P_2	64,016,001	64,039,999	24,000
g_9	8,001	P_3	64,016,001	96,035,999	32,020,000
g_{10}	12,001	P_1	144,024,001	432,024,000	288,000,001
g_{11}	12,001	P_2	144,024,001	144,059,999	36,000
g_{12}	12,001	P_3	144,024,001	216,053,999	72,030,000

Table 1
Graphs Data for Different Patterns and Sizes

Graph ID	GPU-HE Time (s)	GPU-Wedge Time (s)	Speedup of GPU-HE	CPU-HE Time (s)	Speedup of GPU-HE	CPU-Wedge Time (s)	Speedup of GPU-HE	CGAL Time (s)	Speedup of GPU-HE	BGL Time (s)	Speedup of GPU-HE
g_1	0.0126	0.0126	1.00	0.315	25.00	1.032	81.90	5.01	397.62	1.55	123.02
g_2	0.00675	0.00715	1.06	0.155	22.96	0.469	69.48	2.395	354.81	1.112	164.74
g_3	0.0088	0.0098	1.11	0.223	25.34	0.651	73.98	3.027	343.98	1.69	192.05
g_4	0.398	0.4316	1.08	5.207	13.08	18.093	45.46	92.46	232.31	N/A	N/A
g_5	0.13	0.142	1.09	3.31	25.46	8.83	67.92	42.93	330.23	N/A	N/A
g_6	0.179	0.195	1.09	4.69	26.20	12.74	71.17	56.43	315.25	N/A	N/A
g_7	1.577	1.72	1.09	21.13	13.40	75.26	47.72	417.035	264.45	N/A	N/A
g_8	0.496	0.551	1.11	15.578	31.41	39.9	80.44	179.22	361.33	N/A	N/A
g_9	0.77	0.84	1.09	20.85	27.08	56.0	72.73	243.38	316.08	N/A	N/A
g_{10}	3.52	3.85	1.09	47.99	13.63	173.29	49.23	1026.41	291.59	N/A	N/A
g_{11}	1.09	1.23	1.13	36.15	33.17	97.82	89.74	441.28	404.84	N/A	N/A
g_{12}	1.7	1.89	1.11	47.9	28.18	131.42	77.31	612.0	360.00	N/A	N/A

Table 2
Experimental results for regions extraction of the different graphs using the different implementations and the speedup of the HE-based GPU implementation time against the different implementations.

the secondary wedge-sorting step required by the wedge algorithm, with the latter typically being up to 13% slower across test cases. Consequently, the primary comparison focuses on this CPU implementation and the CUDA-based GPU implementation for the HE-based algorithm which are the fastest CPU and GPU implementations.

Figure 11 illustrates the resulting speedup, demonstrating a clear increase in performance gains as the graph size grows. The speedup reaches up to 33× for the largest graph instances, with an average speedup of 23.7×. These results highlight the significant performance benefits of the GPU-based implementation. Pattern 1 exhibits lower speedup due to significant load imbalance caused by its non-uniform region distribution. The algorithm's parallel efficiency depends on the maximum region size; since Pattern 1 is dominated by one massive exterior region (with size $4(l+1)$) alongside many small triangular regions, GPU threads must wait for the largest region to process. High speedup is more readily achieved in patterns where region sizes are homogeneous, ensuring a balanced workload.

6. Conclusion

In this work, we proposed two efficient GPU-accelerated algorithms for region extraction in plane graphs. The practical implementations, developed in CUDA C++, are optimized through techniques such as pointer jumping and by leveraging established CUDA libraries, including the Thrust library. The 2 original sequential algorithms have the same run-time complexity $O(m \log m)$, but the HE-based one achieves better performance than the Wedge-based and between the two proposed GPU-accelerated algorithms, the algorithm that is based on the half-edges achieves better run-time than the Wedge-based algorithm. The fastest proposed approach achieves up to 33× speedup compared to the fastest CPU-based implementations, enabling the extraction of planar graph regions for large-scale instances.

The GPU-accelerated method has broad applicability across multiple domains, including computer graphics, geographic information systems (GIS), VLSI circuit design, and robotics. Its advantages are particularly pronounced when

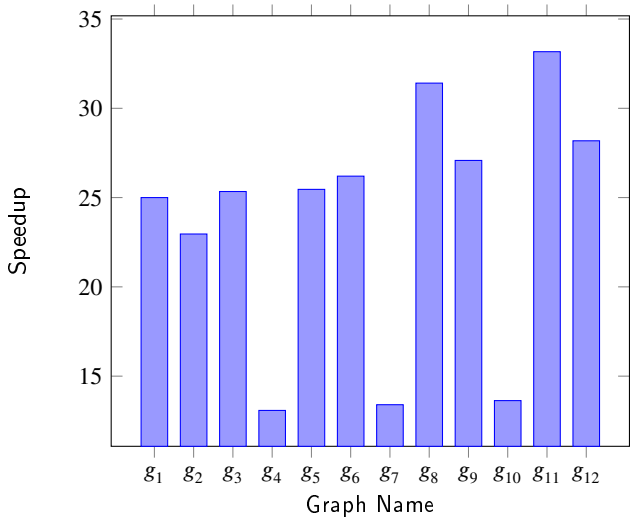


Figure 11: Speedup between the fastest CPU and the GPU implementations on different graphs (the HE-based implementations)

processing extremely large graphs containing substantial numbers of vertices and edges.

References

[1] , 2002. The Boost Graph Library: user guide and reference manual. Addison-Wesley Longman Publishing Co., Inc., USA.

[2] Antolin, P., Buffa, A., Cirillo, E., 2023. Region extraction in mesh intersection. Computer-Aided Design 156, 103448. URL: https://www.sciencedirect.com/science/article/pii/S0010448522001816, doi:https://doi.org/10.1016/j.cad.2022.103448.

[3] Bell, N., Hoberock, J., 2012. Chapter 26 - thrust: A productivity-oriented library for cuda, in: mei W. Hwu, W. (Ed.), GPU Computing Gems Jade Edition. Morgan Kaufmann, Boston. Applications of GPU Computing Series, pp. 359–371. URL: https://www.sciencedirect.com/science/article/pii/B9780123859631000265, doi:https://doi.org/10.1016/B978-0-12-385963-1.00026-5.

[4] Cheng, P., Mao, L.F., Shen, W.H., Yan, Y.L., 2025. Electromigration failures in integrated circuits: A review of physics-based models and analytical methods. Electronics 14. URL: https://www.mdpi.com/2079-9292/14/15/3151, doi:10.3390/electronics14153151.

[5] De Berg, M., Van Kreveld, M., Overmars, M., Cheong, O., 2008. Computational Geometry: Algorithms and Applications. 3rd ed., Springer-Verlag Berlin Heidelberg. doi:10.1007/978-3-540-77974-2.

[6] de Magalhães, S.V.G., Franklin, W.R., Andrade, M.V.A., 2020. An efficient and exact parallel algorithm for intersecting large 3-d triangular meshes using arithmetic filters. Computer-Aided Design 120, 102801. URL: https://www.sciencedirect.com/science/article/pii/S0010448519305330, doi:https://doi.org/10.1016/j.cad.2019.102801.

[7] Diestel, R., 2025. Planar Graphs. Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 93–122. URL: https://doi.org/10.1007/978-3-662-70107-2_4, doi:10.1007/978-3-662-70107-2_4.

[8] Fabri, A., Pion, S., 2009. Cgal: the computational geometry algorithms library, in: Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, Association for Computing Machinery, New York, NY, USA. p. 538–539. URL: https://doi.org/10.1145/1653771.1653865, doi:10.1145/1653771.1653865.

[9] JaJá, J., 1992. An Introduction to Parallel Algorithms. Addison-Wesley.

[10] Jiang, X., Bunke, H., 1993. An optimal algorithm for extracting the regions of a plane graph. Pattern Recognition Letters 14, 553–558. URL: https://www.sciencedirect.com/science/article/pii/016786559390104L, doi:https://doi.org/10.1016/0167-8655(93)90104-L.

[11] Lintermann, A., 2021. Computational Meshing for CFD Simulations. Springer Singapore, Singapore. pp. 85–115. URL: https://doi.org/10.1007/978-981-15-6716-2_6, doi:10.1007/978-981-15-6716-2_6.

[12] Lintermann, A., Schlimpert, S., Grimmen, J., Günther, C., Meinke, M., Schröder, W., 2014. Massively parallel grid generation on hpc systems. Computer Methods in Applied Mechanics and Engineering 277, 131–153. URL: https://www.sciencedirect.com/science/article/pii/S0045782514001340, doi:https://doi.org/10.1016/j.cma.2014.04.009.

[13] Magalhães, S.V.G., Franklin, W.R., Andrade, M.V.A., 2017. Fast exact parallel 3d mesh intersection algorithm using only orientation predicates, in: Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, Association for Computing Machinery, New York, NY, USA. URL: https://doi.org/10.1145/3139958.3140001, doi:10.1145/3139958.3140001.

[14] Nickolls, J., Buck, I., Garland, M., Skadron, K., 2008. Scalable parallel programming with cuda, in: ACM SIGGRAPH 2008 Classes, Association for Computing Machinery, New York, NY, USA. URL: https://doi.org/10.1145/1401132.1401152, doi:10.1145/1401132.1401152.